



CS-2001
Data Structures
Fall 2023

Graph Implementation

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Add Vertex (VertexType vertex)

```
template<class vertexType>
void GraphType<vertexType>::AddVertex(vertexType
vertex)
{
    vertices[numVertices] = vertex;
    for (int index = 0; index <= numVertices; index++)
    {
        edge[numVertices][index] = NULL_EDGE;
        edge[index][numVertices] = NULL_EDGE;
    }
    numVertices++;
}
```

AddEdge(VertexType toVertex, VertexType fromVertex, int weight)

```
template<class vertexType>
void GraphType<vertexType>::AddEdge(vertexType
fromVertex, vertexType toVertex, int weight)
{
    int row, col;
    row = IndexIs(vertices, fromVertex);
    col = IndexIs(vertices, toVertex);
    edge[row][col] = weight;
}
```

IndexIs(vertexType* Vertices , vertexType Vertex)

```
template<class vertexType>
int IndexIs(vertexType* vertices, vertexType
vertex)
{
    int index = 0;
    while (!(vertex == vertices[index]))
        index++;
    return index;
}
```

Return weight of edge

```
template<class vertexType>
int GraphType<vertexType>::WeightIs(vertexType
fromVertex, vertexType toVertex)
{
    int row, col;
    row = IndexIs(vertices, fromVertex);
    col = IndexIs(vertices, toVertex);
    return edge[row][col];
}
```

Adjacent Vertices

```
template<class vertexType>
void GraphType<vertexType>::GetToVertices(vertexType
vertex, QueType& adjVertices)
{
    int fromIndex;
    int toIndex;

    fromIndex = IndexIs(vertices, vertex);
    for (toIndex = 0; toIndex < numVertices; toIndex++)
        if (edges[fromIndex][toIndex] != NULL_EDGE)
            adjVertices.Enqueue(vertices[toIndex]);
}
```

Representing Graphs:

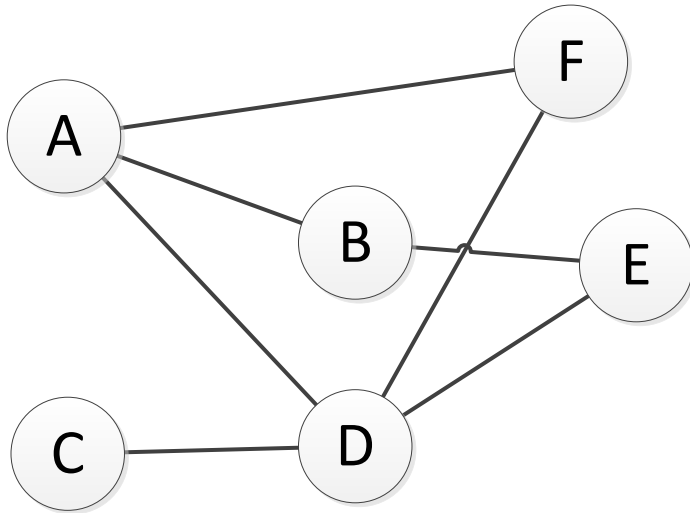
Adjacency List

Adjacency List

- Array of lists
- Each vertex has an array entry
- A vertex **w** is inserted in the list for vertex **v** if there is an outgoing edge from **v** to **w**

Adjacency matrix Representation

An undirected graph $G = (V, E)$

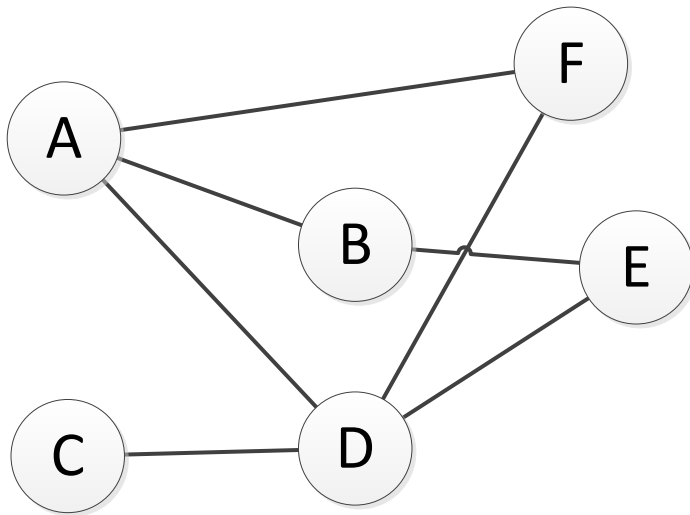


Adjacency matrix for G

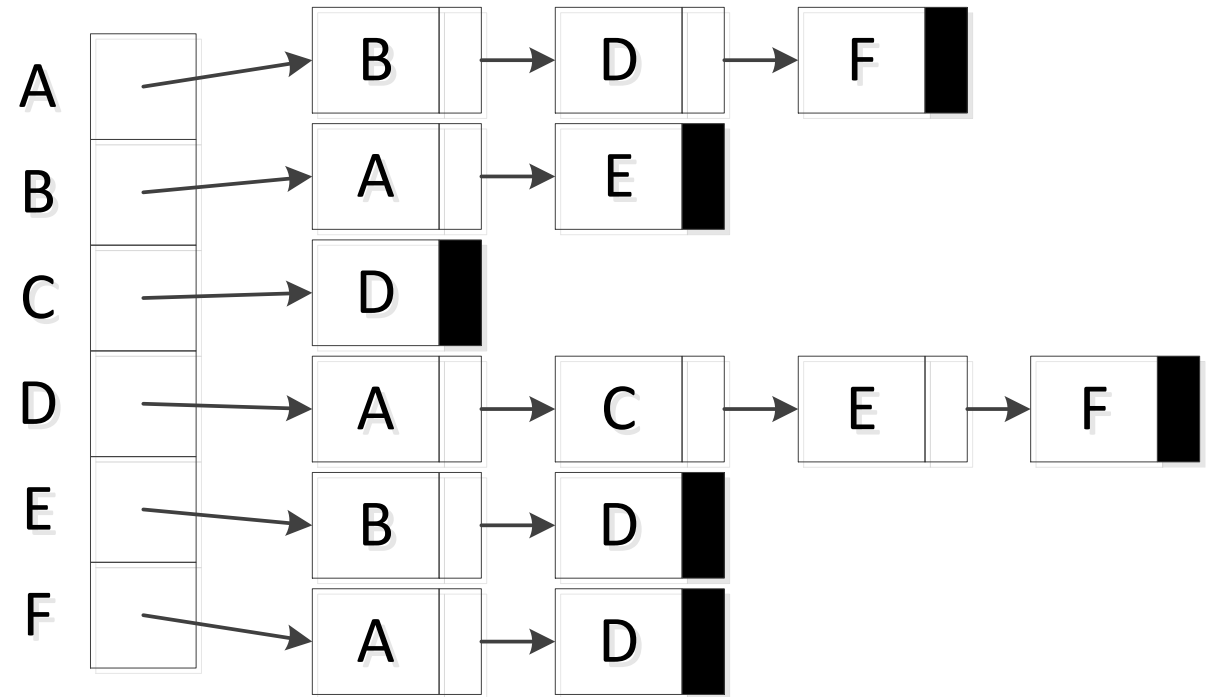
	A	B	C	D	E	F
A	0	1	0	1	0	1
B	1	0	0	0	1	0
C	0	0	0	1	0	0
D	1	0	1	0	1	1
E	0	1	0	1	0	0
F	1	0	0	1	0	0

Adjacency List Representation

An undirected graph $G = (V, E)$



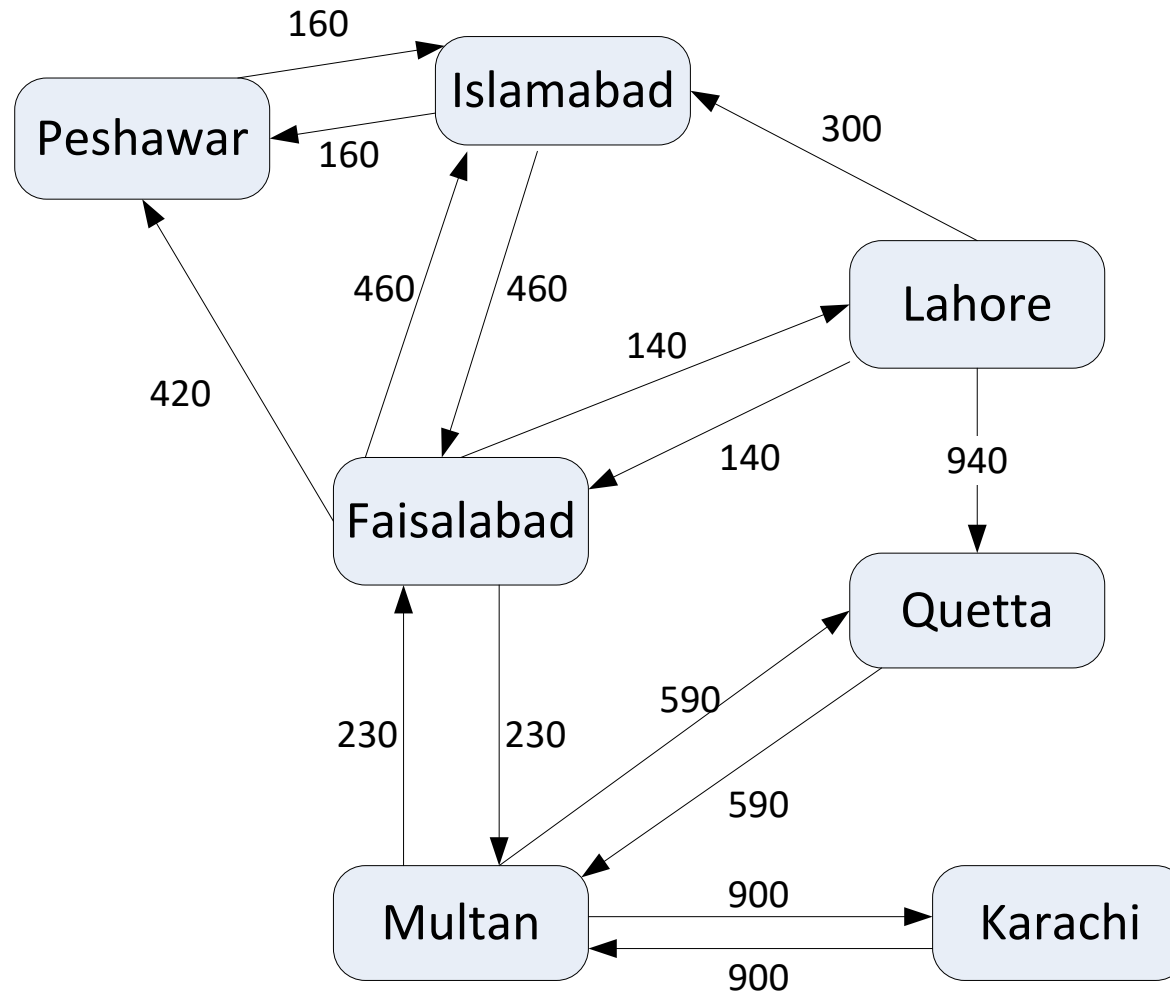
Adjacency List for G



Adjacency Lists Representation

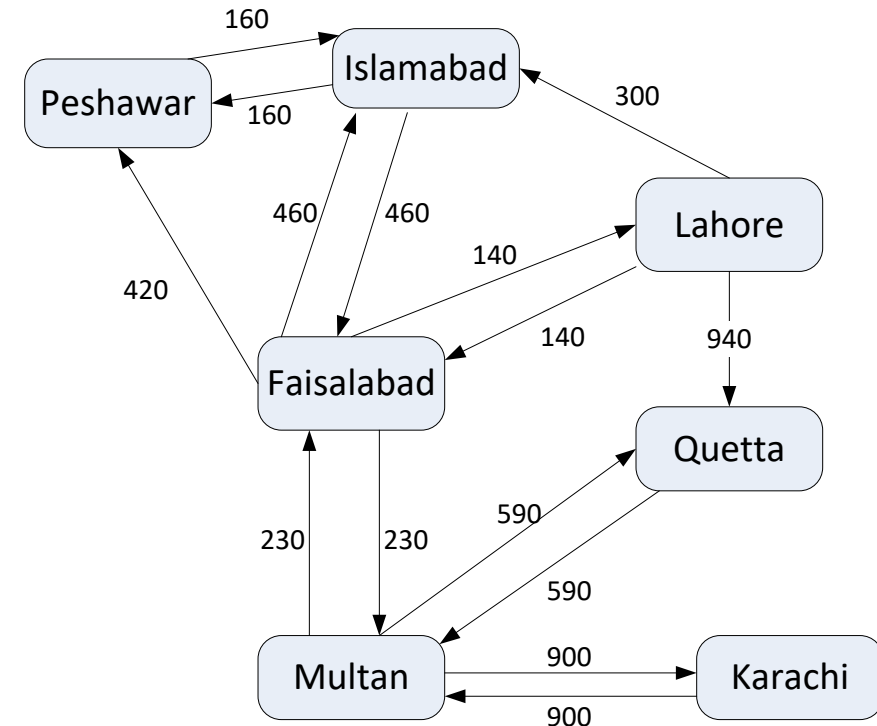
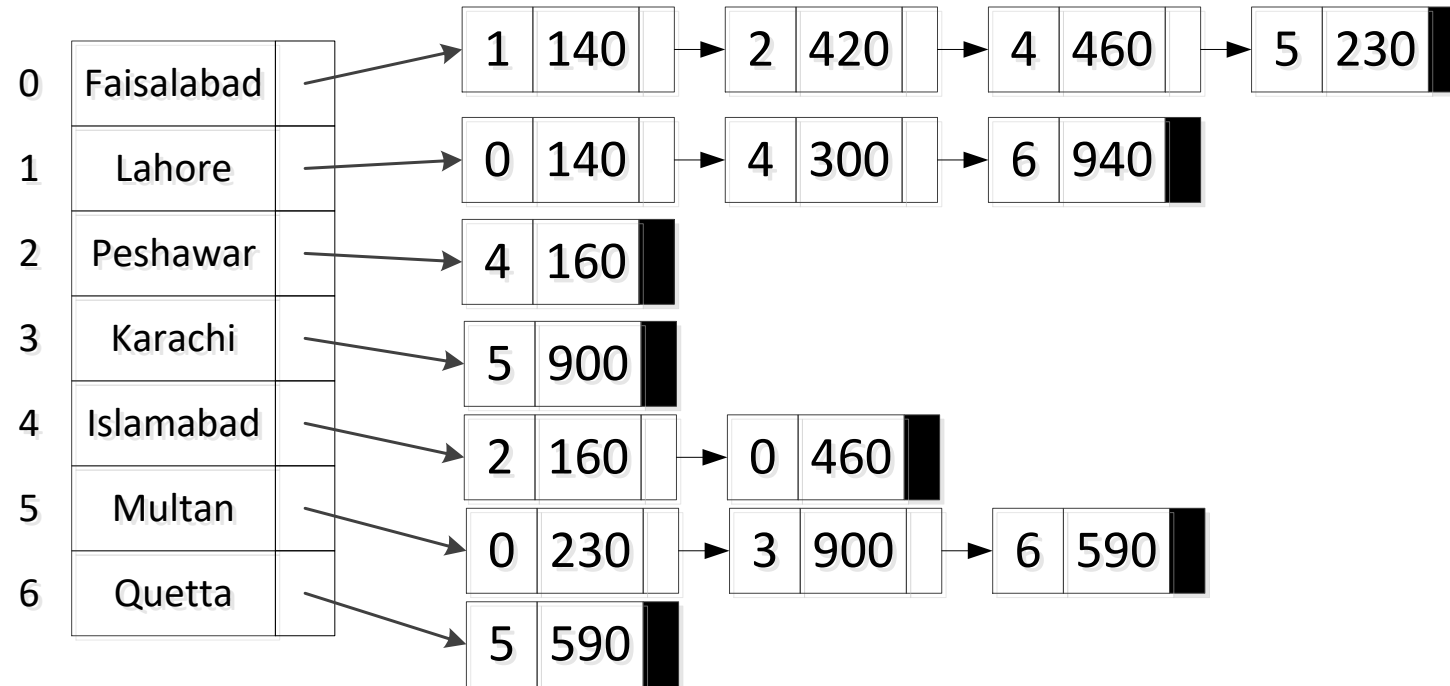
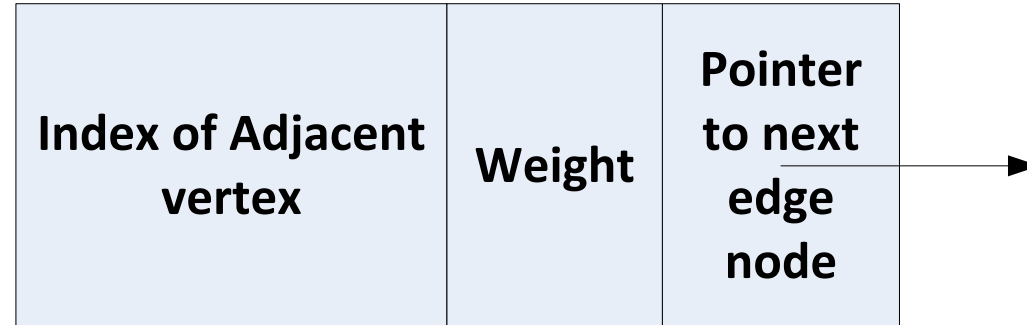
- A graph of n nodes is represented by a one-dimensional array L of linked lists, where
 - $L[i]$ is the linked list containing all the nodes adjacent to node i
 - The nodes in the list $L[i]$ are in no particular order
 - An adjacency list for a weighted graph should contain two elements in the list nodes – one element for the vertex and the second element for the weight of that edge

Weighted directed graph



Adjacency List –Weighted graph Example

Edge nodes



Pros and Cons of Adjacency Matrix

- **Pros:**

- Simple to implement
- Easy and fast to tell if a pair (i,j) is an edge: simply check if $A[i][j]$ is 1 or 0

- **Cons:**

- No matter how few edges the graph has, the matrix takes $O(n^2)$ in memory

Pros and Cons of Adjacency Lists

- **Pros:**

- Saves on space (memory): the representation takes as many memory as there are nodes and edge.

- **Cons:**

- It can take up to **$O(n)$** time to determine if a pair of nodes **(i,j)** is an edge: one would have to search the linked list **$L[i]$** , which takes time proportional to the length of **$L[i]$** .

Reading Assignment

- Other methods of representing a graph
 - Write a small document explaining Adjacency Set with an example.